THE PRACTICAL HANDBOOK · 2026 EDITION

Git for DevOps

Every Git command used in the daily DevOps life cycle, explained step by step in simple language, with flowcharts and real terminal examples.



30 commands explained

17 flowcharts & diagrams

48 terminal sessions

20 interview Q&A

10 hands-on labs

Written by

Ajay Upadhyay

DevOps Engineer

From zero to production-ready

How to Use This Book

This book takes you from **"What is Git?"** to using it confidently in real DevOps work: CI/CD pipelines, Pull Requests, releases, rollbacks and GitOps. Read the chapters in order the first time. Later, use the **Cheat Sheet (Ch. 15)** and **Troubleshooting (Ch. 14)** as quick references.

Every command is explained the same way

WHAT IT DOES
The purpose of the command in one or two simple sentences.

HOW TO USE IT
The syntax, with the most common forms you will actually type.

USEFUL OPTIONS
The flags worth knowing, and nothing more.

SEE IT IN THE TERMINAL
A realistic terminal session with the real output you will see.

Boxes you will see

🧠 **Simple Way to Remember**: an easy analogy

⚙️ **In DevOps**: how it's used in real pipelines and teams

💡 **Pro Tip**: a habit that saves time

⚠️ **Careful**: things that can lose work or break production

Reading the terminal examples

example

```
ajay@devops:~/webapp$ git status  # the command you type is white
On branch main  # output from Git is grey
+ added lines are green
- removed lines are red
```

The prompt `ajay@devops:~/webapp$` shows the user and the current folder. You type only what comes after the `$`. All examples use a sample project called **webapp** belonging to a company called **techcorp**.

Contents

1 **What is Git & Why DevOps Needs It**
Before typing any command, let's understand the big picture: what problem Git solves and why every DevOps pipeline starts with it.

2 **Git Vocabulary in Plain English**
Git has its own language. Here is every important word explained simply, the way you would explain it to a friend.

3 **Installing & Configuring Git**
Five minutes of setup that you only do once per machine. Do it right and every commit you make will be properly signed with your name.

4 **How Git Works: The 4 Areas**
Git can feel confusing because changes live in different 'places'. Once you can picture these places, Git stops being magic.

5 **Everyday Commands: Create, Save, Inspect**
The essential commands to start a repo, save your work and look at history. Master these and you can already work on real projects.

6 **Undoing Mistakes Safely**
Committed the wrong thing? Broke production? Here is how to undo changes at every stage, and which methods are safe on shared branches.

7 **Branching & Merging**
Branches are Git's superpower. They let a whole team work in parallel without stepping on each other.

8 **Working with Remotes (GitHub / GitLab)**
Git becomes a team tool the moment you connect it to a remote. Here's how to push, pull and fetch, and how to connect securely with SSH.

9 **Power Tools Every DevOps Engineer Uses**
Stash, rebase, cherry-pick, tags, reflog and bisect: the tools that save the day during releases and incidents.

10 **.gitignore: Keeping Secrets & Junk Out**
The cheapest security tool in DevOps is a good .gitignore file. One wrong commit of a secret can become a real security incident.

11 **Branching Strategies Used in Real Teams**
GitFlow, GitHub Flow or Trunk-Based? Every company picks one. Here's how each works and when to choose it.

12 **Git in the CI/CD Pipeline & GitOps**
This is where Git meets DevOps. Learn how a simple git push becomes a tested, deployed application.

13 **A Day in the Life: Git in a DevOps Job**
Everything you've learned, used together in one realistic working day, including an urgent production hotfix.

14 **Troubleshooting: Common Errors & Fixes**
Error messages look scary, but most Git problems have a simple, well-known fix.

15 **Git Cheat Sheet**
Every important command on two pages. Perfect for printing, pinning or sharing.

16 **Top 20 Git Interview Questions for DevOps**
The questions that come up again and again in DevOps, SRE and Cloud interviews, with short, clear answers.

17 **Hands-On Practice Labs**
Ten practical exercises, from your first commit to a tagged release. Perfect as homework or a classroom workshop.

CHAPTER 1

What is Git & Why DevOps Needs It

Before typing any command, let's understand the big picture: what problem Git solves and why every DevOps pipeline starts with it.

IN THIS CHAPTER YOU WILL LEARN

- What version control means, in plain words
- The difference between Git and GitHub
- Centralized vs distributed systems
- Why Git is the foundation of CI/CD and GitOps

1.1 The problem Git solves

Imagine you are writing a project report. You save **report.docx**, then **report_v2.docx**, then **report_final.docx**, then **report_final_REALLY_final.docx**. Now imagine 20 people editing the same files at the same time. Who changed what? Which version is correct? How do you go back to last Tuesday's version?

This mess is exactly what a **Version Control System (VCS)** fixes. It records every change, who made it, when, and why, so you can always go back in time.

> 🧠 **Simple Way to Remember**
>
> Think of Git like the **save points in a video game**. Every time you reach a good point, you save (a *commit*). If something goes wrong later, you load an earlier save. And you can even play two different paths at the same time (*branches*).

1.2 So, what is Git?

Git is a free, open-source, distributed version control system. It was created in **2005 by Linus Torvalds** (the creator of Linux) to manage the Linux kernel source code. Today it is the standard tool used by nearly every software and DevOps team in the world.

- **Version control**: it remembers every version of every file.
- **Distributed**: every developer has a full copy of the entire history on their own machine, not only the latest files.
- **Fast & offline**: most commands run locally, so they are instant and work without internet.

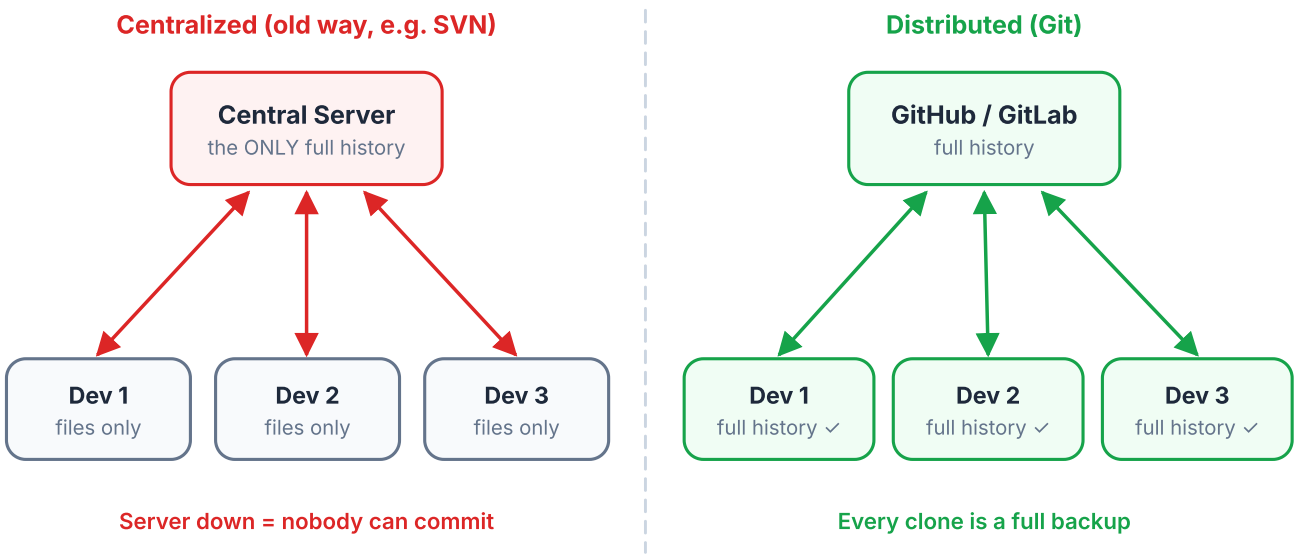


Figure 1. Centralized vs Distributed version control. In Git, every developer has the complete history.

1.3 Git is NOT GitHub

This is the #1 confusion for beginners. **Git** is the tool that runs on your computer. **GitHub**, **GitLab**, **Bitbucket** and **Azure Repos** are websites that *host* Git repositories online so teams can share them, review code and run pipelines.

	Git	GitHub / GitLab / Bitbucket
What it is	A command-line tool (software)	A website / cloud service
Where it runs	Your laptop, servers, CI agents	On the internet (or company servers)
Main job	Track changes and history	Store repos, Pull Requests, CI/CD, issues
Needs internet?	No (mostly works offline)	Yes
Analogy	A camera that takes photos	Instagram, where you share the photos

1.4 Why Git is the heart of DevOps

In DevOps, **everything is code**: application code, Dockerfiles, Kubernetes YAML, Terraform, Ansible playbooks, pipeline files. All of it lives in Git. That makes Git the **single source of truth**.

DevOps need	How Git helps
Automation (CI/CD)	A `git push` automatically triggers build, test and deploy pipelines.
Collaboration	Branches + Pull Requests let many people work safely in parallel.
Rollback	Something broke in production? Revert to the last good commit in seconds.

DevOps need	How Git helps
Audit & compliance	Every change records who, what, when and why.
Infrastructure as Code	Servers and cloud resources are versioned just like application code.
GitOps	Tools like Argo CD / Flux watch a Git repo and make the cluster match it.

CHAPTER 2

Git Vocabulary in Plain English

Git has its own language. Here is every important word explained simply, the way you would explain it to a friend.

IN THIS CHAPTER YOU WILL LEARN

- The 18 terms you will hear every day
- What HEAD, branch and commit really are
- A simple mental model to remember them

Learn these words once and every Git conversation becomes easy. Come back to this page whenever a word feels unclear.

Term	Simple meaning
Repository (repo)	A project folder that Git is tracking. It contains your files plus a hidden `.git` folder that stores the full history.
Commit	A saved snapshot of your project at one moment, with a message explaining the change. Like a save point.
SHA / Commit ID	A unique 40-character ID for every commit, e.g. `a1b2c3d4…`. Usually you only need the first 7 characters.
Branch	An independent line of work. You can build a feature on a branch without touching the main code.
main / master	The default, primary branch. Usually this is what runs in production.
HEAD	A pointer that says "you are here". Normally it points to the branch you are on.
Working directory	The actual files you see and edit on disk.
Staging area (index)	A waiting room where you put the changes you want in your next commit.
Remote	A copy of the repository hosted somewhere else (GitHub, GitLab...).
origin	The default nickname for the remote you cloned from.
Clone	Download a full copy of a remote repository to your machine.
Push / Pull	Push = send your commits to the remote. Pull = get new commits from the remote.

Term	Simple meaning
Merge	Combine the changes of one branch into another.
Pull Request (PR) / Merge Request (MR)	A request on GitHub/GitLab asking teammates to review and merge your branch.
Conflict	When two people changed the same line differently, Git needs a human to decide.
Tag	A permanent label on a commit, usually for releases like `v1.2.0`.
Fork	Your own server-side copy of someone else's repo (common in open source).
.gitignore	A file listing things Git should never track (secrets, logs, build output).

2.1 How branches and HEAD look inside Git



Figure 2. A branch is just a movable label on a commit. HEAD tells Git where YOU are right now.

> 🧠 **Simple Way to Remember**
>
> A **commit** is a photo. A **branch** is a sticky note on the newest photo of an album. **HEAD** is your finger pointing to the album you are looking at right now.

CHAPTER 3

Installing & Configuring Git

Five minutes of setup that you only do once per machine. Do it right and every commit you make will be properly signed with your name.

IN THIS CHAPTER YOU WILL LEARN

- How to install Git on Linux, macOS and Windows
- Setting your name, email and default branch
- The three config levels: system, global, local
- Creating time-saving aliases

3.1 Install Git

Git usually comes pre-installed on Linux servers and CI runners. On your own machine, install it with your package manager:

bash — ajay@devops: ~

```
# Ubuntu / Debian
ajay@devops:~$ sudo apt update && sudo apt install git -y
# RHEL / CentOS / Amazon Linux / Fedora
ajay@devops:~$ sudo dnf install git -y
# macOS (with Homebrew)
ajay@devops:~$ brew install git
# Windows: download "Git for Windows" from git-scm.com (includes Git Bash)
ajay@devops:~$ git --version
git version 2.47.1
```

3.2 Tell Git who you are (one-time setup)

Every commit stores the author's name and email. Set them once, before your first commit. Use the same email as your GitHub/GitLab account so commits are linked to your profile.

#01 git config

WHAT IT DOES

Reads and writes Git settings: your identity, default branch name, editor, aliases and more.

HOW TO USE IT

```
git config --global user.name  "Your Name"
git config --global user.email "you@company.com"
git config --list                # see all settings
git config user.name             # read one setting
```

USEFUL OPTIONS

Option	Meaning
`--global`	Applies to all repos for your user (stored in `~/.gitconfig`)
`--local`	Applies only to the current repo (default; stored in `.git/config`)
`--system`	Applies to every user on the machine
`--list --show-origin`	Show every setting and which file it comes from
`--unset`	Remove a setting

SEE IT IN THE TERMINAL

bash — ajay@devops: ~

```
ajay@devops:~$ git config --global user.name "Ajay Upadhyay"
ajay@devops:~$ git config --global user.email "ajay@techcorp.com"
ajay@devops:~$ git config --global init.defaultBranch main  # new repos start on
'main'
ajay@devops:~$ git config --global core.editor "vim"
ajay@devops:~$ git config --global pull.rebase false  # 'git pull' will merge
(default)
ajay@devops:~$ git config --list --global
user.name=Ajay Upadhyay
user.email=ajay@techcorp.com
init.defaultbranch=main
core.editor=vim
pull.rebase=false
```

⚙️ **In DevOps**

On shared CI servers or bots, set identity with `--local` inside the pipeline (e.g. `git config user.name "ci-bot"`) so automated commits are clearly marked as coming from the pipeline.

3.3 Handy aliases (save keystrokes)

Aliases are your own shortcuts. These are popular with DevOps engineers:

```
bash — ajay@devops: ~/webapp

ajay@devops:~/webapp$ git config --global alias.st status
ajay@devops:~/webapp$ git config --global alias.co checkout
ajay@devops:~/webapp$ git config --global alias.lg "log --oneline --graph --all --decorate"
ajay@devops:~/webapp$ git lg
* 3f9c2e1 (HEAD -> main, origin/main) Add health-check endpoint
* 7b1d0aa Update Dockerfile base image
* 1a2b3c4 Initial commit
```

3.4 Getting help

```
bash — ajay@devops: ~/webapp

ajay@devops:~/webapp$ git help commit  # full manual page
ajay@devops:~/webapp$ git commit -h  # short summary of options
```

> 💡 **Pro Tip**
>
> Line endings: on Windows run `git config --global core.autocrlf true`; on Linux/macOS use `input`. This avoids "every line changed" diffs when teams mix operating systems.

CHAPTER 4

How Git Works: The 4 Areas

Git can feel confusing because changes live in different 'places'. Once you can picture these places, Git stops being magic.

IN THIS CHAPTER YOU WILL LEARN

- Working directory, staging area, local and remote repo
- The four states of a file
- Why the staging area exists
- The Edit → Add → Commit → Push loop

This is the **most important chapter** in the book. If you understand these four areas, every Git command will make sense, because almost every command simply moves changes from one area to another.

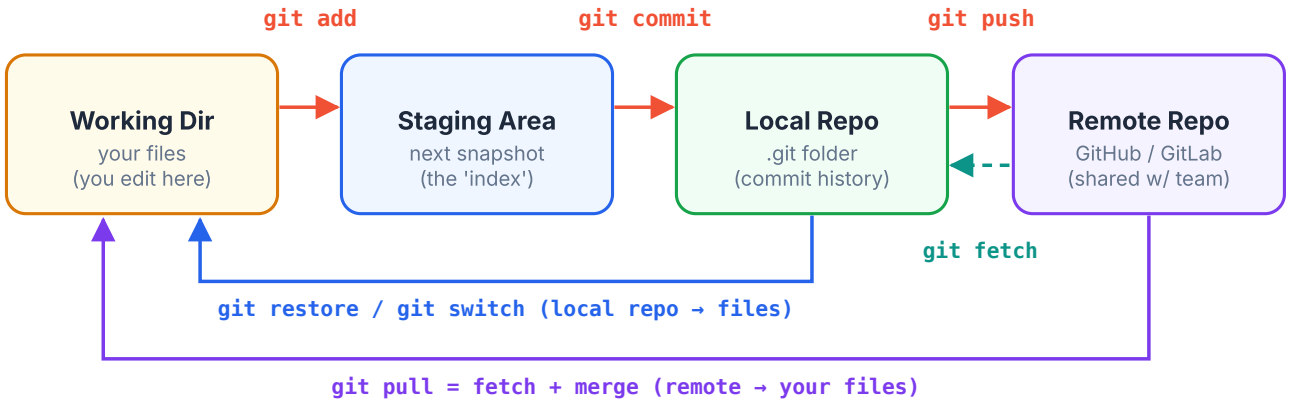


Figure 3. The 4 places your code lives, and the commands that move it between them.

Area	What it is	Real-life analogy
Working Directory	The files on your disk that you edit	Your desk, where you are writing
Staging Area	Changes you've selected for the next commit	The envelope you are packing
Local Repository	All saved commits, in the `.git` folder	Your own filing cabinet
Remote Repository	Shared copy on GitHub/GitLab	The company's central archive

4.1 Why do we need a staging area?

Say you changed 5 files: 3 for a bug fix and 2 for a new feature. The staging area lets you commit **only the 3 bug-fix files** first, with a clear message, then commit the other 2 separately. Small, focused commits are much easier to review, revert and debug.

4.2 The life of a file

Every file in your project is always in one of four states:

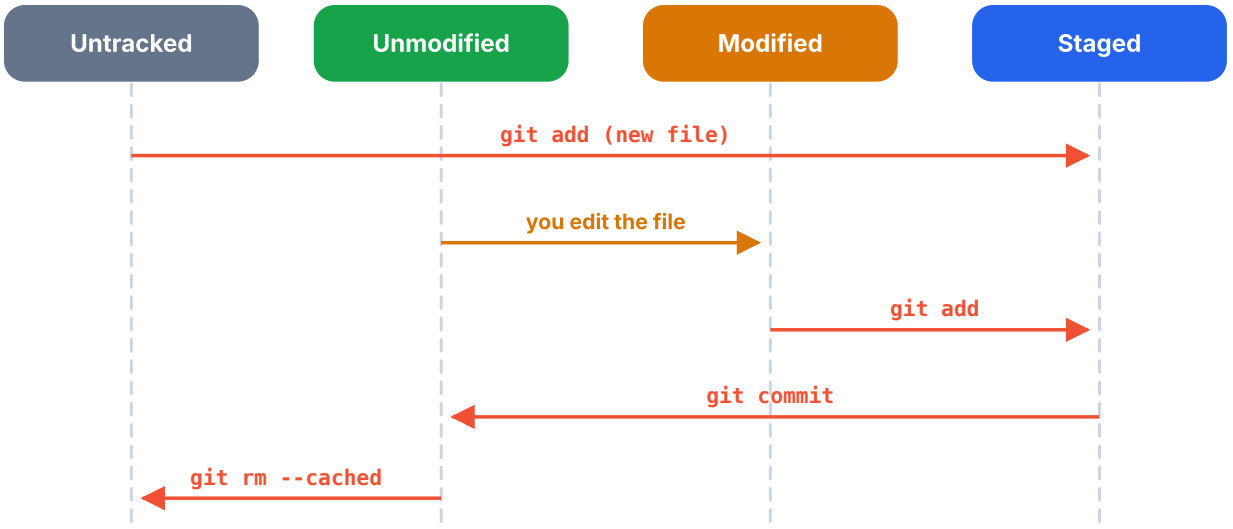


Figure 4. Lifecycle of a file. Each arrow is something you do.

State	Meaning	Shown by git status as
Untracked	New file Git has never seen	Untracked files
Unmodified	Tracked and same as last commit	(not shown, it's clean)
Modified	Changed but not staged yet	Changes not staged for commit
Staged	Ready to go into the next commit	Changes to be committed

> 🧠 **Simple Way to Remember**
>
> **Edit → Add → Commit → Push.**
> Say it out loud like a mantra. 90% of your daily Git work is this loop.

CHAPTER 5

Everyday Commands: Create, Save, Inspect

The essential commands to start a repo, save your work and look at history. Master these and you can already work on real projects.

IN THIS CHAPTER YOU WILL LEARN

- git init, clone, status, add, commit
- Reading history with git log and git show
- Seeing exact line changes with git diff
- How to write commit messages your team will love

These eight commands are the ones you will type every single day. Each card shows what the command does, how to use it, and exactly what you will see in the terminal.

#02 git init

WHAT IT DOES

Turns the current folder into a new Git repository by creating a hidden `.git` folder. Your files are not changed.

WHEN YOU USE IT

Starting a brand-new project that is not on GitHub/GitLab yet.

HOW TO USE IT

```
git init                # in the current folder
git init my-project     # create a new folder and init it
```

USEFUL OPTIONS

Option	Meaning
`-b main`	Name of the first branch (overrides config)
`--bare`	Create a server-side repo with no working files

SEE IT IN THE TERMINAL

```
bash — ajay@devops: ~/webapp

ajay@devops:~$ mkdir webapp && cd webapp
ajay@devops:~/webapp$ git init
Initialized empty Git repository in /home/ajay/webapp/.git/
ajay@devops:~/webapp$ ls -a
.  ..  .git
```

> ⚠️ **Careful**
>
> Never delete or edit the `.git` folder by hand. It *is* your entire history.

#03 git clone

WHAT IT DOES

Downloads a complete copy of a remote repository (all files, all branches, full history) and sets up `origin` automatically.

WHEN YOU USE IT

Joining an existing project. This is how most of your work starts.

HOW TO USE IT

```
git clone <url>
git clone <url> <folder-name>
git clone -b <branch> <url>
```

USEFUL OPTIONS

Option	Meaning
`-b develop`	Check out a specific branch after cloning
`--depth 1`	Shallow clone: only the latest commit. Much faster in CI pipelines
`--recurse-submodules`	Also clone submodules

SEE IT IN THE TERMINAL

```
bash — ajay@devops: ~/webapp

ajay@devops:~$ git clone git@github.com:techcorp/webapp.git
Cloning into 'webapp'...
remote: Enumerating objects: 1243, done.
remote: Counting objects: 100% (1243/1243), done.
remote: Compressing objects: 100% (612/612), done.
Receiving objects: 100% (1243/1243), 2.31 MiB | 4.20 MiB/s, done.
Resolving deltas: 100% (598/598), done.
ajay@devops:~$ cd webapp
```

⚙️ **In DevOps**

CI/CD jobs almost always use `git clone --depth 1` (or a shallow checkout) to save time and bandwidth. A 2 GB history becomes a few MB.

#04 git status

WHAT IT DOES

Shows the current state: which branch you are on, which files are modified, staged or untracked. **Run it all the time.** It is your dashboard.

HOW TO USE IT

```
git status
git status -s      # short format
```

USEFUL OPTIONS

Option	Meaning
`-s / --short`	Compact one-line-per-file view
`-b`	Show branch info in short mode

SEE IT IN THE TERMINAL

```
bash — ajay@devops: ~/webapp

ajay@devops:~/webapp$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   app.py

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        Dockerfile

no changes added to commit (use "git add" and/or "git commit -a")
ajay@devops:~/webapp$ git status -s
 M app.py
?? Dockerfile
```

> 📌 **Note**
> In short format: `??` = untracked, `M` on the right = modified, `M` on the left = staged, `A` = added (new file staged), `D` = deleted.

#05 git add

WHAT IT DOES

Moves changes from the working directory into the **staging area**, so they will be included in the next commit.

HOW TO USE IT

```
git add <file>          # one file
git add <folder>/        # a folder
git add .                # everything in current folder
git add -p               # choose pieces interactively
```

USEFUL OPTIONS

Option	Meaning
`.`	Stage all changes in current directory and below
`-A`	Stage all changes in the whole repo, including deletions

Option	Meaning
`-p`	Review each change (hunk) and choose y/n. Great for clean commits
`-u`	Stage only already-tracked files

SEE IT IN THE TERMINAL

```
bash — ajay@devops: ~/webapp

ajay@devops:~/webapp$ git add Dockerfile app.py
ajay@devops:~/webapp$ git status -s
A  Dockerfile
M  app.py
```

⚙️ **In DevOps**

Before `git add .`, always run `git status`. It's the easiest way to avoid accidentally committing `.env` files, keys or big build folders.

#06 git commit

WHAT IT DOES

Takes everything in the staging area and saves it permanently as a new snapshot (commit) in your local repository, with a message.

HOW TO USE IT

```
git commit -m "message"
git commit                  # opens editor for a longer message
git commit -am "message"    # add tracked files + commit
```

USEFUL OPTIONS

Option	Meaning
`-m "msg"`	Write the commit message inline
`-a`	Automatically stage modified *tracked* files (not new ones)
`--amend`	Fix the last commit (see Chapter 6)
`-S`	GPG-sign the commit (verified badge)

SEE IT IN THE TERMINAL

```
bash — ajay@devops: ~/webapp

ajay@devops:~/webapp$ git commit -m "feat: add Dockerfile for webapp"
[main 4e5f6a7] feat: add Dockerfile for webapp
 2 files changed, 18 insertions(+), 2 deletions(-)
 create mode 100644 Dockerfile
```

💡 **Good vs Bad Messages**

Writing a good commit message: use the imperative mood ("Add", "Fix", not "Added"), keep the first line under ~50 characters, and say *why*, not only what.

✅ `fix: increase nginx timeout to 60s for slow uploads`

❌ `updated stuff`

#07 git log

WHAT IT DOES

Shows the commit history: who committed what, when, and with which message. Newest first.

HOW TO USE IT

```
git log
git log --oneline
git log --oneline --graph --all --decorate
```

USEFUL OPTIONS

Option	Meaning
`--oneline`	One short line per commit
`--graph`	Draw branch/merge lines
`-n 5`	Only the last 5 commits
`--author="Ajay"`	Filter by author
`--since="2 weeks ago"`	Filter by date
`-p`	Show the actual changes (patch)
`-- path/file`	History of one file
`--grep="fix"`	Search commit messages

SEE IT IN THE TERMINAL

```
bash — ajay@devops: ~/webapp

ajay@devops:~/webapp$ git log -n 2
commit 4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f (HEAD -> main)
Author: Ajay Upadhyay <ajay@techcorp.com>
Date:   Sat Oct 24 10:42:11 2026 +0530

    feat: add Dockerfile for webapp

commit 3f9c2e1a0b9c8d7e6f5a4b3c2d1e0f9a8b7c6d5e (origin/main)
Author: Priya Sharma <priya@techcorp.com>
Date:   Fri Oct 23 18:05:37 2026 +0530

    Add health-check endpoint
ajay@devops:~/webapp$ git log --oneline --graph --all
* 4e5f6a7 (HEAD -> main) feat: add Dockerfile for webapp
* 3f9c2e1 (origin/main) Add health-check endpoint
* 7b1d0aa Update Dockerfile base image
* 1a2b3c4 Initial commit
```

⚙️ **In DevOps**

During an incident, `git log --since="6 hours ago" --oneline` on the deployed branch quickly answers the classic question: *"What changed right before it broke?"*

#08 git diff

WHAT IT DOES

Shows exactly which lines changed. Lines starting with **+** were added, lines with **-** were removed.

HOW TO USE IT

```
git diff                  # working dir vs staging
git diff --staged         # staging vs last commit
git diff main..feature    # between two branches
```

USEFUL OPTIONS

Option	Meaning
`--staged`	What you are about to commit (same as --cached)
`--stat`	Summary: files and number of lines changed
`--name-only`	Only file names

Option	Meaning
`<commit1> <commit2>`	Compare any two commits

SEE IT IN THE TERMINAL

```
bash — ajay@devops: ~/webapp

ajay@devops:~/webapp$ git diff
diff --git a/config.yaml b/config.yaml
index 8d3e2f1..a4b5c6d 100644
--- a/config.yaml
+++ b/config.yaml
@@ -3,5 +3,5 @@ server:
   host: 0.0.0.0
-  port: 8080
+  port: 9090
   workers: 4
```

> ⚙️ **In DevOps**
>
> Monorepo pipelines use `git diff --name-only HEAD~1 HEAD` to detect which services changed, and then build and deploy only those.

#09 git show

WHAT IT DOES

Displays the details of one commit: the author, date, message and the full diff of what changed.

HOW TO USE IT

```
git show                 # latest commit (HEAD)
git show <sha>
git show <sha> --stat
```

USEFUL OPTIONS

Option	Meaning
`--stat`	List changed files only
`HEAD~2`	Two commits before HEAD
`<sha>:path/file`	Show a file as it was in that commit

SEE IT IN THE TERMINAL

```
bash — ajay@devops: ~/webapp

ajay@devops:~/webapp$ git show 3f9c2e1 --stat
commit 3f9c2e1a0b9c8d7e6f5a4b3c2d1e0f9a8b7c6d5e
Author: Priya Sharma <priya@techcorp.com>
Date:   Fri Oct 23 18:05:37 2026 +0530

    Add health-check endpoint

 app.py           | 8 ++++++++
 tests/test_app.py | 6 ++++++
 2 files changed, 14 insertions(+)
```

5.1 Putting it all together: your first project

```
bash — ajay@devops: ~/webapp

ajay@devops:~$ mkdir webapp && cd webapp
ajay@devops:~/webapp$ git init
Initialized empty Git repository in /home/ajay/webapp/.git/
ajay@devops:~/webapp$ echo "print('Hello DevOps')" > app.py
ajay@devops:~/webapp$ git status -s
?? app.py
ajay@devops:~/webapp$ git add app.py
ajay@devops:~/webapp$ git commit -m "Initial commit: add app.py"
[main (root-commit) 1a2b3c4] Initial commit: add app.py
 1 file changed, 1 insertion(+)
 create mode 100644 app.py
ajay@devops:~/webapp$ git log --oneline
1a2b3c4 (HEAD -> main) Initial commit: add app.py
```

CHAPTER 6

Undoing Mistakes Safely

Committed the wrong thing? Broke production? Here is how to undo changes at every stage, and which methods are safe on shared branches.

IN THIS CHAPTER YOU WILL LEARN

- Discard or unstage changes with git restore
- Fix the last commit with --amend
- The 3 modes of git reset
- Safe production rollbacks with git revert
- Untracking files and cleaning up

Everybody makes mistakes: a wrong file committed, a typo in a message, a bad deployment. Git can undo almost anything. You just need to pick the right tool.

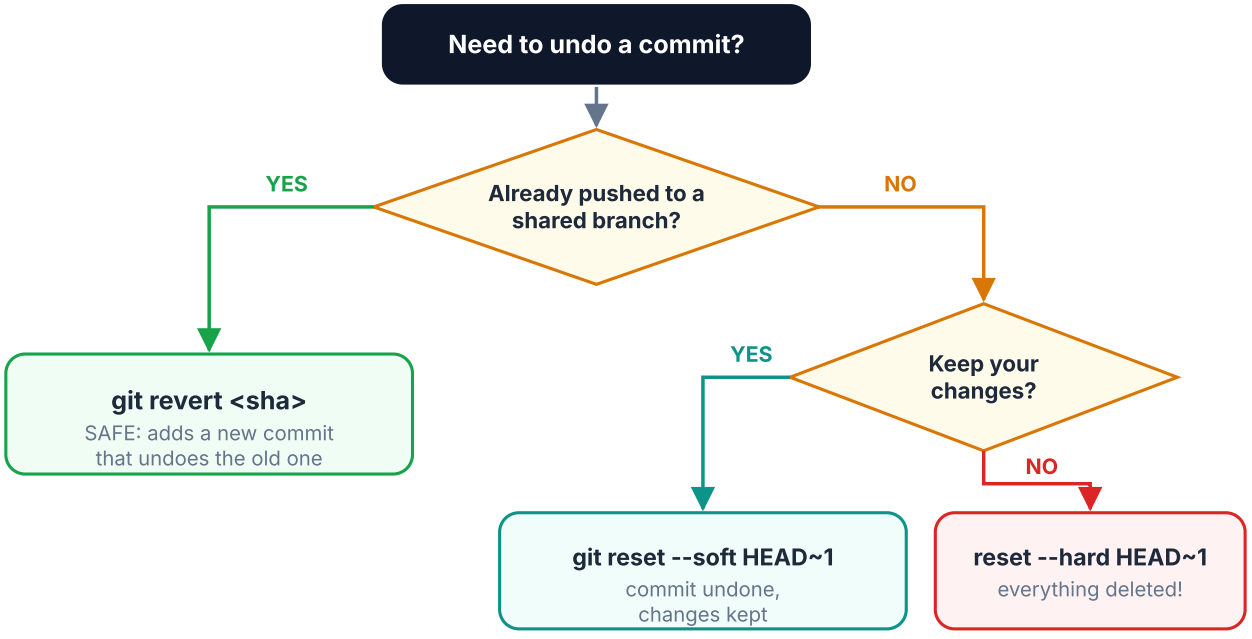


Figure 5. Decision tree: which undo command should you use?

#10 git restore

WHAT IT DOES

Throws away changes in your working directory (back to the last commit), or removes a file from the staging area.

HOW TO USE IT

```
git restore <file>              # discard local changes
git restore --staged <file>     # unstage (keep changes)
git restore --source=<sha> <file>  # bring back an old version
```

USEFUL OPTIONS

Option	Meaning
`--staged`	Unstage only. Your edits are kept
`--source=<sha>`	Restore the file content from a specific commit
`.`	All files

SEE IT IN THE TERMINAL

bash — ajay@devops: ~/webapp

```
ajay@devops:~/webapp$ git status -s
 M nginx.conf
ajay@devops:~/webapp$ git restore nginx.conf  # undo my edits
ajay@devops:~/webapp$ git status -s
ajay@devops:~/webapp$ git add secrets.env
ajay@devops:~/webapp$ git restore --staged secrets.env  # oops, unstage it
ajay@devops:~/webapp$ git status -s
?? secrets.env
```

> ⚠️ **Careful**
>
> `git restore <file>` permanently deletes your uncommitted edits to that file. There is no undo for uncommitted work.

#11 git commit --amend

WHAT IT DOES

Replaces the last commit with a new one. Use it to fix the commit message or add a file you forgot.

HOW TO USE IT

```
git commit --amend -m "new message"
git add forgotten.txt && git commit --amend --no-edit
```

USEFUL OPTIONS

Option	Meaning
`--no-edit`	Keep the same message
`-m`	Change the message

SEE IT IN THE TERMINAL

```
bash — ajay@devops: ~/webapp

ajay@devops:~/webapp$ git commit -m "Add k8s deploymnet"
[main 9a8b7c6] Add k8s deploymnet
ajay@devops:~/webapp$ git add k8s/service.yaml  # forgot this file
ajay@devops:~/webapp$ git commit --amend -m "Add k8s deployment and service"
[main 2d3e4f5] Add k8s deployment and service
 2 files changed, 41 insertions(+)
```

> ⚠️ **Careful**
>
> Only amend commits that you have **not pushed yet**. Amending creates a new commit ID, which breaks history for anyone who already pulled it.

#12 git reset

WHAT IT DOES

Moves your branch pointer back to an earlier commit. Depending on the mode, your changes are kept staged, kept unstaged, or deleted.

HOW TO USE IT

```
git reset --soft  HEAD~1
git reset --mixed HEAD~1    # default
git reset --hard  HEAD~1
```

SEE IT IN THE TERMINAL

```
bash — ajay@devops: ~/webapp

ajay@devops:~/webapp$ git log --oneline -3
c3c3c3c (HEAD -> main) WIP: broken config
b2b2b2b Add redis cache
a1a1a1a Initial commit
ajay@devops:~/webapp$ git reset --soft HEAD~1
ajay@devops:~/webapp$ git log --oneline -2
b2b2b2b (HEAD -> main) Add redis cache
a1a1a1a Initial commit
ajay@devops:~/webapp$ git status -s
M  config.yaml
```

Mode	Commit	Staging area	Your files	Use when
`--soft`	Removed	Kept (staged)	Kept	Redo the commit differently
`--mixed`	Removed	Cleared	Kept	Re-select which files to commit
`--hard`	Removed	Cleared	DELETED	Throw everything away

🧠 **Simple Way to Remember**

`HEAD~1` means "one commit before where I am". `HEAD~3` means three commits back.

⚠️ **Careful**

`git reset --hard` destroys uncommitted work, and resetting commits that are already pushed causes trouble for your team. For shared branches, use `git revert`.

#13 git revert

WHAT IT DOES

Creates a **new commit** that does the exact opposite of an old commit. History is not rewritten, so it is completely safe on shared branches like `main`.

HOW TO USE IT

```
git revert <sha>
git revert HEAD              # undo the latest commit
git revert -m 1 <merge-sha>  # undo a merge commit
```

USEFUL OPTIONS

Option	Meaning
`--no-edit`	Use the default message
`-n`	Revert without committing yet (combine several reverts)
`-m 1`	Required when reverting a merge commit; keeps parent #1 (main)

SEE IT IN THE TERMINAL

bash — ajay@devops: ~/webapp

```
ajay@devops:~/webapp$ git log --oneline -3
f00ba12 (HEAD -> main, origin/main) Upgrade payment SDK to v5
e1e1e1e Add metrics exporter
d0d0d0d Fix login redirect
ajay@devops:~/webapp$ git revert f00ba12 --no-edit
[main 7c7c7c7] Revert "Upgrade payment SDK to v5"
 1 file changed, 1 insertion(+), 1 deletion(-)
ajay@devops:~/webapp$ git push origin main
```

> ⚙️ **In DevOps**
>
> This is the standard **production rollback** in a GitOps setup: revert the bad commit, push, and the pipeline (or Argo CD) automatically redeploys the previous good version, with a clear audit trail.

#14 `git rm / git mv`

WHAT IT DOES

Deletes or renames files *and* tells Git about it in one step.

HOW TO USE IT

```
git rm <file>
git rm --cached <file>     # stop tracking, keep on disk
git mv <old> <new>
```

USEFUL OPTIONS

Option	Meaning
`--cached`	Remove from Git only; the file stays on your disk
`-r`	Recursive, for folders

SEE IT IN THE TERMINAL

```
bash — ajay@devops: ~/webapp

ajay@devops:~/webapp$ git rm --cached .env  # committed by mistake
rm '.env'
ajay@devops:~/webapp$ echo ".env" >> .gitignore
ajay@devops:~/webapp$ git mv deploy.sh scripts/deploy.sh
ajay@devops:~/webapp$ git status -s
D  .env
R  deploy.sh -> scripts/deploy.sh
 M .gitignore
```

> ⚠️ **Careful**
>
> `git rm --cached` stops tracking a secret, but it still exists in **old commits**. If a password or key was ever pushed, **rotate it immediately** (see Chapter 14).

#15 git clean

WHAT IT DOES

Deletes **untracked** files (files Git doesn't know about), such as build output or temp files.

HOW TO USE IT

```
git clean -n      # dry run: show what would be deleted
git clean -fd     # delete untracked files and folders
```

USEFUL OPTIONS

Option	Meaning
`-n`	Dry run. Always do this first!
`-f`	Force (required to actually delete)
`-d`	Include folders
`-x`	Also delete ignored files

SEE IT IN THE TERMINAL

```
bash — ajay@devops: ~/webapp

ajay@devops:~/webapp$ git clean -n
Would remove build/
Would remove debug.log
ajay@devops:~/webapp$ git clean -fd
Removing build/
Removing debug.log
```

CHAPTER 7

Branching & Merging

Branches are Git's superpower. They let a whole team work in parallel without stepping on each other.

IN THIS CHAPTER YOU WILL LEARN

- Creating, listing and deleting branches
- git switch vs the classic git checkout
- Fast-forward vs 3-way merges
- Resolving merge conflicts step by step

A **branch** lets you work on something new (a feature, a bug fix, an experiment) without touching the stable code. When the work is done and reviewed, you **merge** it back.

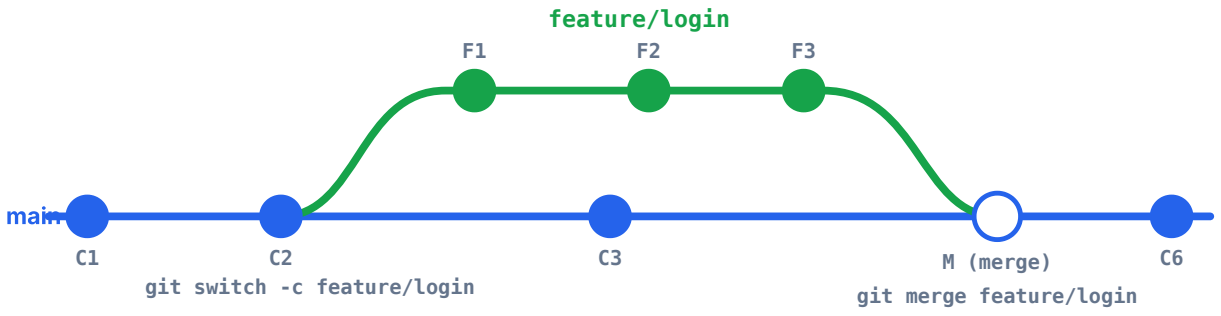


Figure 6. A feature branch splits from main, gets its own commits, then is merged back.

🧠 **Simple Way to Remember**

Branches in Git are extremely cheap: creating one just creates a tiny pointer file. So create a new branch for **every** task, no matter how small.

#16 `git branch`

WHAT IT DOES

Lists, creates, renames and deletes branches.

HOW TO USE IT

```
git branch                 # list local branches
git branch <name>          # create (doesn't switch)
git branch -d <name>       # delete merged branch
git branch -m <old> <new>  # rename
```

USEFUL OPTIONS

Option	Meaning
`-a`	List local + remote branches
`-r`	Only remote branches
`-vv`	Show last commit and tracking info
`-d`	Delete (only if merged)
`-D`	Force delete (even if not merged)
`--merged`	Branches already merged into the current one

SEE IT IN THE TERMINAL

```
bash — ajay@devops: ~/webapp

ajay@devops:~/webapp$ git branch -a
  develop
* main
  remotes/origin/HEAD -> origin/main
  remotes/origin/develop
  remotes/origin/main
ajay@devops:~/webapp$ git branch -vv
  develop 8e8e8e8 [origin/develop] Add helm chart
* main    3f9c2e1 [origin/main] Add health-check endpoint
```

> 📌 **Note**
>
> The `*` marks the branch you are currently on.

#17 git switch

WHAT IT DOES

Moves you to another branch (and updates your files to match it). Use `-c` to create a new branch and switch to it in one step.

HOW TO USE IT

```
git switch <branch>
git switch -c <new-branch>
git switch -          # go back to previous branch
```

USEFUL OPTIONS

Option	Meaning
`-c`	Create and switch
`-`	Switch to the previous branch
`--detach <sha>`	Look at an old commit (detached HEAD)

SEE IT IN THE TERMINAL

```
bash — ajay@devops: ~/webapp

ajay@devops:~/webapp$ git switch -c feature/add-redis-cache
Switched to a new branch 'feature/add-redis-cache'
ajay@devops:~/webapp$ git branch
* feature/add-redis-cache
  main
ajay@devops:~/webapp$ git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
```

> ⚙️ **In DevOps**
>
> Use clear branch names with a prefix: `feature/`, `bugfix/`, `hotfix/`, `release/`, `chore/`. Many teams also add the ticket ID: `feature/OPS-142-redis-cache`. Pipelines often use these prefixes to decide what to build and deploy.

#18 git checkout

WHAT IT DOES

The older all-in-one command. It can switch branches *and* restore files. Since Git 2.23 its jobs were split into `git switch` and `git restore`, but you will still see it everywhere in tutorials and scripts.

HOW TO USE IT

```
git checkout <branch>         # = git switch <branch>
git checkout -b <new-branch>  # = git switch -c
git checkout -- <file>        # = git restore <file>
```

SEE IT IN THE TERMINAL

```
bash — ajay@devops: ~/webapp

ajay@devops:~/webapp$ git checkout -b bugfix/nginx-timeout
Switched to a new branch 'bugfix/nginx-timeout'
```

#19 git merge

WHAT IT DOES

Combines another branch into the branch you are currently on.

HOW TO USE IT

```
git switch main
git merge <branch>
```

USEFUL OPTIONS

Option	Meaning
`--no-ff`	Always create a merge commit (keeps a visible record of the feature)
`--squash`	Squash all branch commits into one change, then you commit
`--abort`	Cancel a merge that has conflicts

SEE IT IN THE TERMINAL

```
bash — ajay@devops: ~/webapp

ajay@devops:~/webapp$ git switch main
ajay@devops:~/webapp$ git merge feature/add-redis-cache
Updating 3f9c2e1..9d9d9d9
Fast-forward
 cache/redis.py     | 34 ++++++++++++++++++++++++++++++++++
 requirements.txt   |  1 +
 2 files changed, 35 insertions(+)
ajay@devops:~/webapp$ git branch -d feature/add-redis-cache
Deleted branch feature/add-redis-cache (was 9d9d9d9).
```

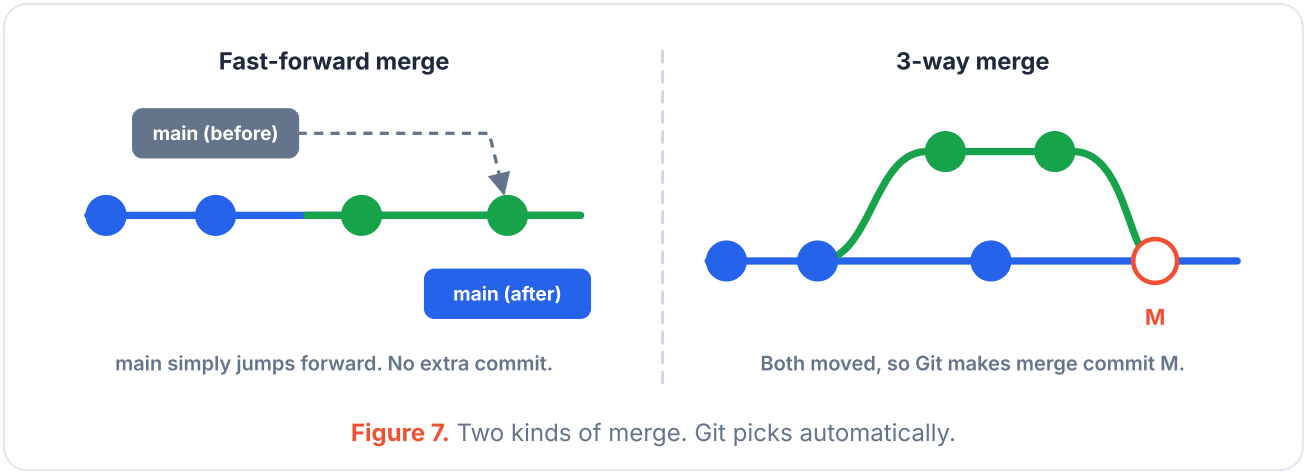


Figure 7. Two kinds of merge. Git picks automatically.

7.1 Merge conflicts: don't panic!

A **conflict** happens when two branches changed **the same lines** of the same file. Git can't guess which version is right, so it asks you. This is normal, not an error.

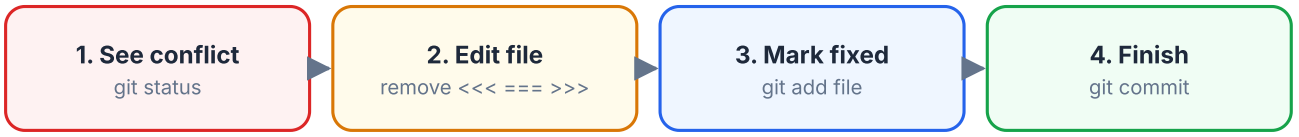


Figure 8. Resolving a merge conflict in 4 steps.

bash — ajay@devops: ~/webapp

```
ajay@devops:~/webapp$ git merge feature/update-port
Auto-merging config.yaml
CONFLICT (content): Merge conflict in config.yaml
Automatic merge failed; fix conflicts and then commit the result.
ajay@devops:~/webapp$ git status -s
UU config.yaml
ajay@devops:~/webapp$ cat config.yaml
server:
<<<<<<< HEAD
  port: 8080
=======
  port: 9090
>>>>>>> feature/update-port
```

1. **Read the markers.**
   Everything between `<<<<<<< HEAD` and `=======` is **your current branch**. Between `=======` and `>>>>>>>` is **the incoming branch**.

2. **Edit the file.**
   Keep what is correct (one side, the other, or a mix) and **delete all three marker lines**.

3. **Mark as resolved.**
   `git add config.yaml`

4. **Finish the merge.**
   `git commit` (Git pre-fills the message). Changed your mind? `git merge --abort` returns to before the merge.

```
bash — ajay@devops: ~/webapp

ajay@devops:~/webapp$ vim config.yaml  # keep port 9090, remove markers
ajay@devops:~/webapp$ git add config.yaml
ajay@devops:~/webapp$ git commit --no-edit
[main 5b5b5b5] Merge branch 'feature/update-port'
```

> 💡 **Pro Tip**
>
> Prevent conflicts: pull from `main` often, keep branches short-lived, and keep Pull Requests small. VS Code and most IDEs show buttons like *Accept Current / Accept Incoming / Accept Both* that make resolving much easier.

CHAPTER 8

Working with Remotes (GitHub / GitLab)

Git becomes a team tool the moment you connect it to a remote. Here's how to push, pull and fetch, and how to connect securely with SSH.

IN THIS CHAPTER YOU WILL LEARN

- Adding and inspecting remotes
- Pushing branches and handling rejected pushes
- The real difference between fetch and pull
- Setting up SSH key authentication

So far everything happened on your machine. Now let's connect to GitHub/GitLab and share work with the team.

#20 `git remote`

WHAT IT DOES

Manages the list of remote repositories your local repo knows about. `origin` is just a nickname for a URL.

HOW TO USE IT

```
git remote -v
git remote add origin <url>
git remote set-url origin <new-url>
git remote remove <name>
```

USEFUL OPTIONS

Option	Meaning
`-v`	Show names with URLs
`add`	Add a new remote
`set-url`	Change URL (e.g. switch HTTPS → SSH)
`rename`	Rename a remote

SEE IT IN THE TERMINAL

```
bash — ajay@devops: ~/webapp

ajay@devops:~/webapp$ git remote add origin git@github.com:techcorp/webapp.git
ajay@devops:~/webapp$ git remote -v
origin  git@github.com:techcorp/webapp.git (fetch)
origin  git@github.com:techcorp/webapp.git (push)
```

> 📌 **Note**
> When you fork a project, you usually have two remotes: `origin` (your fork) and `upstream` (the original project).

#21 git push

WHAT IT DOES

Uploads your local commits to the remote repository so others (and the CI/CD pipeline) can see them.

HOW TO USE IT

```
git push
git push -u origin <branch>      # first push of a new branch
git push origin --delete <branch>
git push origin <tag>
```

USEFUL OPTIONS

Option	Meaning
`-u`	Set upstream: after this, plain `git push` / `git pull` work for this branch
`--force-with-lease`	Safer force push: refuses if someone else pushed in the meantime
`--tags`	Push all tags
`--delete`	Delete a remote branch

SEE IT IN THE TERMINAL

```
bash — ajay@devops: ~/webapp

ajay@devops:~/webapp$ git push -u origin feature/add-redis-cache
Enumerating objects: 7, done.
Counting objects: 100% (7/7), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 1.02 KiB | 1.02 MiB/s, done.
Total 4 (delta 2), reused 0 (delta 0), pack-reused 0
remote:
remote: Create a pull request for 'feature/add-redis-cache' on GitHub by visiting:
remote:      https://github.com/techcorp/webapp/pull/new/feature/add-redis-cache
remote:
To github.com:techcorp/webapp.git
 * [new branch]      feature/add-redis-cache -> feature/add-redis-cache
branch 'feature/add-redis-cache' set up to track 'origin/feature/add-redis-cache'.
```

> ⚠️ **Careful**
>
> Never use `git push --force` on `main` or any shared branch. It can erase your teammates' work.
>
> If you really must force push your own branch (e.g. after a rebase), use `--force-with-lease`.

When push is rejected

```
bash — ajay@devops: ~/webapp

ajay@devops:~/webapp$ git push
To github.com:techcorp/webapp.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'github.com:techcorp/webapp.git'
hint: Updates were rejected because the remote contains work that you do
hint: not have locally. This is usually caused by another repository pushing
hint: to the same ref. You may want to first integrate the remote changes
hint: (e.g., 'git pull ...') before pushing again.
ajay@devops:~/webapp$ git pull  # get teammates' commits first
ajay@devops:~/webapp$ git push  # now it works
```

This just means a teammate pushed first. Pull their changes, then push again.

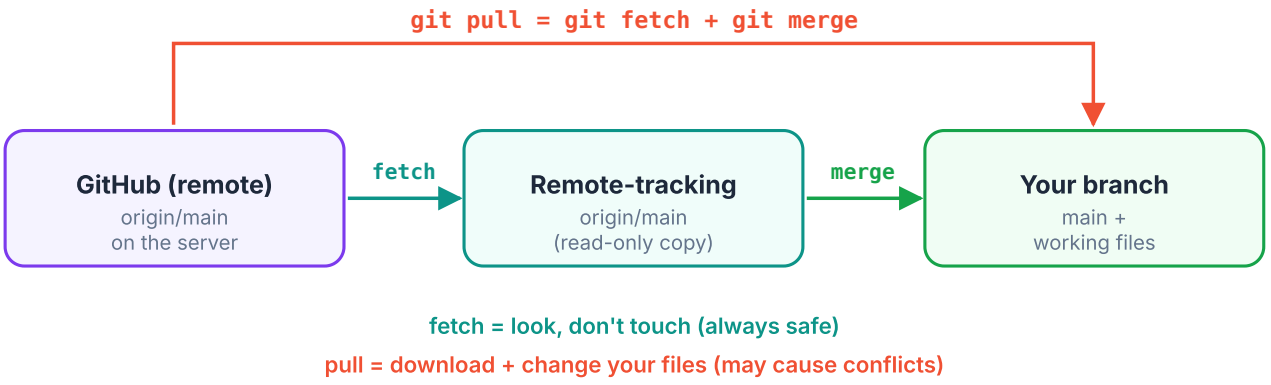


Figure 9. git fetch only downloads. git pull downloads AND updates your branch.

#22 git fetch

WHAT IT DOES

Downloads new commits and branches from the remote **without changing your files**. It just updates your view of what's on the server.

HOW TO USE IT

```
git fetch
git fetch origin
git fetch --prune
```

USEFUL OPTIONS

Option	Meaning
`--prune`	Remove local references to branches deleted on the remote
`--all`	Fetch from all remotes
`--tags`	Fetch all tags

SEE IT IN THE TERMINAL

```
bash — ajay@devops: ~/webapp

ajay@devops:~/webapp$ git fetch --prune
remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (9/9), done.
remote: Total 5 (delta 3), reused 5 (delta 3), pack-reused 0
Unpacking objects: 100% (5/5), 1.18 KiB | 604.00 KiB/s, done.
From github.com:techcorp/webapp
   3f9c2e1..8a8a8a8  main       -> origin/main
 - [deleted]         (none)     -> origin/feature/old-login
ajay@devops:~/webapp$ git log --oneline main..origin/main  # what's new on the server?
8a8a8a8 Bump terraform aws provider to 5.x
```

#23 git pull

WHAT IT DOES

Downloads new commits from the remote **and** merges them into your current branch. It is `git fetch` + `git merge` in one command.

HOW TO USE IT

```
git pull
git pull origin main
git pull --rebase
```

USEFUL OPTIONS

Option	Meaning
`--rebase`	Replay your local commits on top of the remote ones (cleaner history, no extra merge commit)
`--ff-only`	Only update if it's a simple fast-forward; otherwise stop

SEE IT IN THE TERMINAL

```
bash — ajay@devops: ~/webapp

ajay@devops:~/webapp$ git pull
Updating 3f9c2e1..8a8a8a8
Fast-forward
 terraform/providers.tf | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
```

> ⚙️ **In DevOps**
>
> Start every working day and every new branch with `git switch main && git pull`. Working on old code is the #1 cause of painful conflicts.

8.1 Connecting securely: SSH keys

There are two ways to talk to GitHub/GitLab: **HTTPS** (with a Personal Access Token as the password) or **SSH** (with a key pair). DevOps engineers usually prefer SSH: no passwords, and it works great on servers.

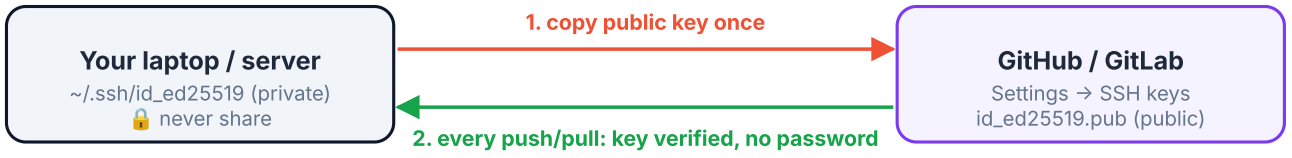


Figure 10. SSH keys: the private key stays on your machine, the public key goes to GitHub.

```
bash — ajay@devops: ~/webapp

ajay@devops:~$ ssh-keygen -t ed25519 -C "ajay@techcorp.com"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/ajay/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Your identification has been saved in /home/ajay/.ssh/id_ed25519
Your public key has been saved in /home/ajay/.ssh/id_ed25519.pub
ajay@devops:~$ cat ~/.ssh/id_ed25519.pub  # copy this to GitHub → Settings → SSH keys
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIK3... ajay@techcorp.com
ajay@devops:~$ ssh -T git@github.com
Hi ajay-upadhyay! You've successfully authenticated, but GitHub does not provide shell
access.
```

> ⚙️ **In DevOps**
>
> For servers and pipelines, don't use your personal key. Use a **deploy key** (read-only key for one repo) or a CI token, and store it in your CI tool's secret store (Jenkins Credentials, GitHub Secrets, GitLab CI variables, Vault).

CHAPTER 9

Power Tools Every DevOps Engineer Uses

Stash, rebase, cherry-pick, tags, reflog and bisect: the tools that save the day during releases and incidents.

IN THIS CHAPTER YOU WILL LEARN

- Pausing work with git stash
- Clean history with rebase (and when NOT to use it)
- Moving single fixes with cherry-pick
- Release versioning with tags
- Recovering 'lost' work with reflog
- Hunting bugs with blame and bisect

These commands turn you from a Git user into the person your team calls when something goes wrong.

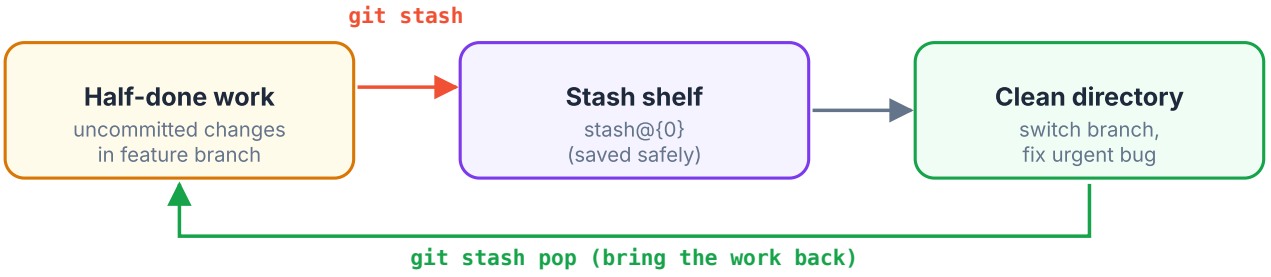


Figure 11. git stash: put half-done work on a shelf, fix something else, then take it back.

#24 git stash

WHAT IT DOES

Temporarily saves your uncommitted changes on a 'shelf' and gives you a clean working directory. Later you can bring them back.

WHEN YOU USE IT

Your manager says "Production is down, drop everything!" while you're in the middle of something.

HOW TO USE IT

```
git stash                       # save changes
git stash push -m "message"
git stash list
git stash pop                   # restore + remove from shelf
git stash apply stash@{1}       # restore, keep on shelf
git stash drop stash@{0}
```

USEFUL OPTIONS

Option	Meaning
`-u`	Also stash untracked (new) files
`pop`	Apply the latest stash and delete it
`apply`	Apply but keep it in the list
`clear`	Delete ALL stashes

SEE IT IN THE TERMINAL

```
bash — ajay@devops: ~/webapp

ajay@devops:~/webapp$ git stash push -m "half-done redis config"
Saved working directory and index state On feature/redis: half-done redis config
ajay@devops:~/webapp$ git switch main  # go fix the urgent production bug...
ajay@devops:~/webapp$ git switch feature/redis
ajay@devops:~/webapp$ git stash list
stash@{0}: On feature/redis: half-done redis config
ajay@devops:~/webapp$ git stash pop
On branch feature/redis
Changes not staged for commit:
        modified:   config/redis.yaml
Dropped refs/stash@{0} (6f6f6f6f6f6f6f6f6f6f6f6f6f6f6f6f6f6f6f6f)
```

#25 git rebase

WHAT IT DOES

Moves (replays) your branch's commits on top of another branch, so it looks as if you started your work from the latest code. The result is a clean, straight history.

HOW TO USE IT

```
git switch feature/x
git rebase main
git rebase -i HEAD~3      # interactive: squash/edit/reorder
```

USEFUL OPTIONS

Option	Meaning
`-i`	Interactive: squash, reword, drop or reorder commits
`--continue`	Continue after fixing a conflict
`--abort`	Cancel and go back to before the rebase

SEE IT IN THE TERMINAL

```
bash — ajay@devops: ~/webapp

ajay@devops:~/webapp$ git switch feature/add-redis-cache
ajay@devops:~/webapp$ git rebase main
Successfully rebased and updated refs/heads/feature/add-redis-cache.
ajay@devops:~/webapp$ git push --force-with-lease
```

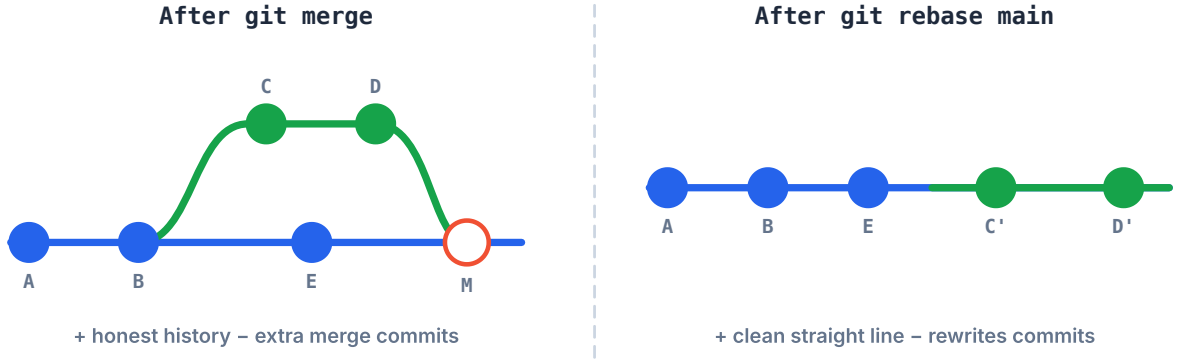


Figure 12. Same work, two histories. Merge keeps the true shape; rebase makes it a straight line.

Interactive rebase: squash 3 messy commits into 1

```
bash — ajay@devops: ~/webapp

ajay@devops:~/webapp$ git rebase -i HEAD~3
# Your editor opens:
pick a1a1a1a Add redis cache
squash b2b2b2b fix typo
squash c3c3c3c fix typo again
# save & close → write one clean message → done
Successfully rebased and updated refs/heads/feature/add-redis-cache.
```

> ⚠️ **Careful**
>
> **The golden rule of rebase:** never rebase commits that others are already using (like `main`). Rebase only your own local or personal feature branches.

#26 git cherry-pick

WHAT IT DOES

Copies one specific commit from another branch and applies it to your current branch.

WHEN YOU USE IT

A bug fix was merged into `main`, and you need the same fix in an older release branch without bringing all the other new features.

HOW TO USE IT

```
git cherry-pick <sha>
git cherry-pick <sha1> <sha2>
```

USEFUL OPTIONS

Option	Meaning
`-x`	Add "(cherry picked from commit ...)" to the message for traceability
`--abort`	Cancel on conflict
`-n`	Apply changes without committing

SEE IT IN THE TERMINAL

```
bash — ajay@devops: ~/webapp

ajay@devops:~/webapp$ git switch release/2.4
ajay@devops:~/webapp$ git cherry-pick -x 7e7e7e7
[release/2.4 1f1f1f1] Fix memory leak in worker
 Date: Thu Oct 22 15:12:40 2026 +0530
 1 file changed, 3 insertions(+), 1 deletion(-)
```

#27 git tag

WHAT IT DOES

Puts a permanent, human-friendly label on a commit, usually a release version like `v1.4.0`. Unlike branches, tags don't move.

HOW TO USE IT

```
git tag                              # list
git tag -a v1.4.0 -m "Release 1.4.0"   # annotated tag
git push origin v1.4.0
git tag -d v1.4.0                    # delete local
```

USEFUL OPTIONS

Option	Meaning
`-a`	Annotated tag: stores author, date and message (recommended for releases)
`-l "v1.*"`	List tags matching a pattern
`--tags`	With push: push all tags
`git describe --tags`	Show the nearest tag, e.g. v1.4.0-3-g9a9a9a9

SEE IT IN THE TERMINAL

```
bash — ajay@devops: ~/webapp

ajay@devops:~/webapp$ git tag -a v1.4.0 -m "Release 1.4.0: redis cache + health checks"
ajay@devops:~/webapp$ git push origin v1.4.0
To github.com:techcorp/webapp.git
 * [new tag]         v1.4.0 -> v1.4.0
ajay@devops:~/webapp$ git tag -l "v1.*"
v1.3.0
v1.3.1
v1.4.0
```

> ⚙️ **In DevOps**
>
> Many release pipelines are triggered by tags: pushing `v1.4.0` builds the Docker image `webapp:1.4.0` and deploys it to production. Follow **Semantic Versioning**: MAJOR.MINOR.PATCH (breaking.feature.fix).

#28 git reflog

WHAT IT DOES

Your safety net. Records every place HEAD has been in the last ~90 days, even commits that seem "lost" after a bad reset or rebase.

HOW TO USE IT

```
git reflog
git reset --hard HEAD@{2}       # go back to that point
git branch rescue <sha>         # or save it as a branch
```

SEE IT IN THE TERMINAL

```
bash — ajay@devops: ~/webapp

ajay@devops:~/webapp$ git reset --hard HEAD~3  # oops! lost 3 commits
HEAD is now at a1a1a1a Initial commit
ajay@devops:~/webapp$ git reflog
a1a1a1a (HEAD -> main) HEAD@{0}: reset: moving to HEAD~3
d4d4d4d HEAD@{1}: commit: Add monitoring dashboards
c3c3c3c HEAD@{2}: commit: Add alert rules
b2b2b2b HEAD@{3}: commit: Add prometheus config
ajay@devops:~/webapp$ git reset --hard d4d4d4d  # 🎉 all 3 commits are back
HEAD is now at d4d4d4d Add monitoring dashboards
```

🧠 **Simple Way to Remember**

If you ever think "I just lost my work in Git", stop and run `git reflog`. If it was ever committed, it's almost certainly still there.

#29 git blame

WHAT IT DOES

Shows who last changed each line of a file, in which commit, and when. (Use it to find context, not to blame people!)

HOW TO USE IT

```
git blame <file>
git blame -L 10,20 <file>      # only lines 10-20
```

USEFUL OPTIONS

Option	Meaning
`-L 10,20`	Limit to a range of lines
`-e`	Show emails instead of names

SEE IT IN THE TERMINAL

```
bash — ajay@devops: ~/webapp

ajay@devops:~/webapp$ git blame -L 3,5 nginx.conf
8e8e8e8e (Priya Sharma  2026-09-14 11:20:03 +0530 3)     listen 80;
5c5c5c5c (Ajay Upadhyay 2026-10-02 16:45:10 +0530 4)     client_max_body_size 50M;
8e8e8e8e (Priya Sharma  2026-09-14 11:20:03 +0530 5)     proxy_read_timeout 60s;
```

#30 git bisect

WHAT IT DOES

Finds the exact commit that introduced a bug using a binary search. You mark commits as good or bad, and Git cuts the search in half each time.

HOW TO USE IT

```
git bisect start
git bisect bad                 # current version is broken
git bisect good v1.3.0         # this version worked
# test, then: git bisect good / git bisect bad
git bisect reset
```

USEFUL OPTIONS

Option	Meaning
`run <script>`	Fully automatic: Git runs your test script on each step

SEE IT IN THE TERMINAL

```
bash — ajay@devops: ~/webapp

ajay@devops:~/webapp$ git bisect start
ajay@devops:~/webapp$ git bisect bad
ajay@devops:~/webapp$ git bisect good v1.3.0
Bisecting: 6 revisions left to test after this (roughly 3 steps)
[9a9a9a9] Refactor DB connection pool
ajay@devops:~/webapp$ git bisect bad
Bisecting: 2 revisions left to test after this (roughly 2 steps)
ajay@devops:~/webapp$ git bisect good
...
4b4b4b4 is the first bad commit
ajay@devops:~/webapp$ git bisect reset
```

⚙️ In DevOps

100 commits between good and bad? Bisect finds the culprit in about 7 steps. With `git bisect run ./test.sh` it does it all by itself.

CHAPTER 10

.gitignore: Keeping Secrets & Junk Out

The cheapest security tool in DevOps is a good .gitignore file. One wrong commit of a secret can become a real security incident.

IN THIS CHAPTER YOU WILL LEARN

- .gitignore pattern rules
- A complete template for DevOps projects
- How to untrack files that were committed by mistake

A `.gitignore` file tells Git which files to **never track**. Put it in the root of your repository and commit it, so the whole team shares the same rules.

Pattern	Meaning	Example match
`*.log`	Any file ending in .log	app.log, logs/error.log
`build/`	A folder named build (anywhere)	build/, src/build/
`/config.local`	Only in the repo root	config.local (not sub/config.local)
`**/temp`	temp in any directory depth	a/b/temp
`!keep.log`	Exception: DO track this	keep.log
`# comment`	A comment line	-

10.1 A ready-to-use DevOps .gitignore

.gitignore

```
# ---------- Secrets (NEVER commit) ----------
.env
*.pem
*.key
id_rsa*
credentials.json
# ---------- Terraform ----------
.terraform/
*.tfstate
*.tfstate.backup
*.tfvars
# ---------- Build / dependencies ----------
node_modules/
target/
dist/
build/
__pycache__/
*.pyc
# ---------- Logs & OS junk ----------
*.log
.DS_Store
Thumbs.db
# ---------- IDE ----------
.vscode/
.idea/
# ---------- Exception: keep this one ----------
!example.env
```

10.2 Ignoring a file that's already tracked

`.gitignore` only affects **untracked** files. If a file was already committed, you need to untrack it first:

bash — ajay@devops: ~/webapp

```
ajay@devops:~/webapp$ echo "*.tfstate" >> .gitignore
ajay@devops:~/webapp$ git rm --cached terraform.tfstate
rm 'terraform.tfstate'
ajay@devops:~/webapp$ git commit -m "chore: stop tracking terraform state"
ajay@devops:~/webapp$ git check-ignore -v terraform.tfstate  # why is it ignored?
.gitignore:9:*.tfstate  terraform.tfstate
```

⚠️ **Careful**

Terraform state files can contain passwords and keys in plain text. Store state in a remote backend (S3 + DynamoDB lock, Terraform Cloud, Azure Storage, GCS), never in Git.

💡 **Pro Tip**

Ready-made templates for every language and tool are at **github.com/github/gitignore**. Also look at **pre-commit** hooks with secret scanners like **gitleaks** to block secrets before they are ever committed.

CHAPTER 11

Branching Strategies Used in Real Teams

GitFlow, GitHub Flow or Trunk-Based? Every company picks one. Here's how each works and when to choose it.

IN THIS CHAPTER YOU WILL LEARN

- How GitFlow works, branch by branch
- GitHub Flow and Trunk-Based Development
- Choosing the right strategy
- Conventional Commit messages

A branching strategy is the team's agreement on **how branches are named, created and merged**. The right one depends on how often you release.

11.1 GitFlow

Two permanent branches: `main` (always equals production) and `develop` (the next release). Work happens in `feature/*` branches; `release/*` branches are used to stabilize; `hotfix/*` branches patch production urgently.

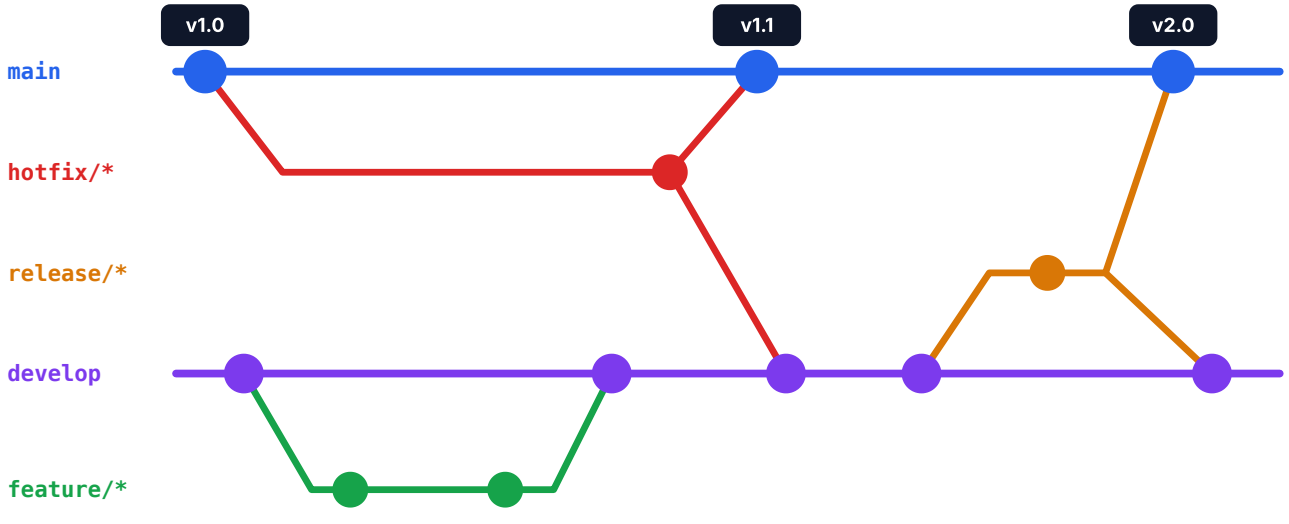


Figure 13. GitFlow: long-lived main & develop, plus feature, release and hotfix branches.

11.2 GitHub Flow (feature branch workflow)

The simplest model and very popular: `main` is always deployable. Create a branch, open a Pull Request, get it reviewed, merge, and deploy. That's it. Chapter 12 shows this flow in detail.

11.3 Trunk-Based Development

Everyone integrates into `main` (the "trunk") at least once a day using very short-lived branches. Unfinished work is hidden behind **feature flags**. This is what high-performing teams doing continuous deployment typically use.

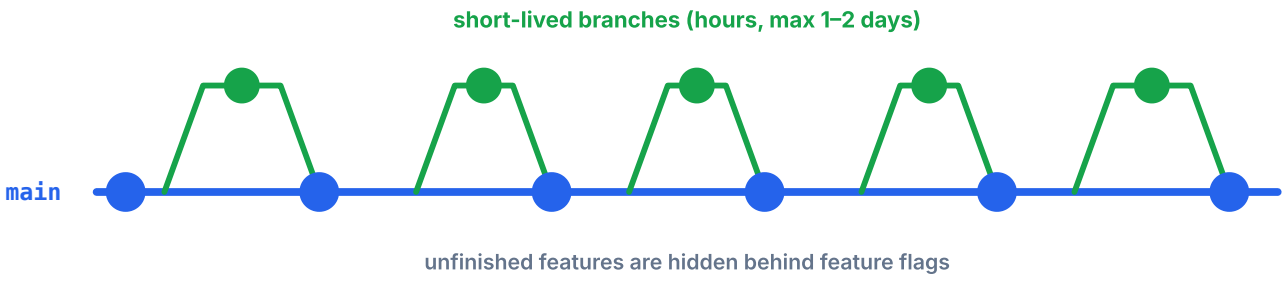


Figure 14. Trunk-based development: everyone merges small changes into main many times a day.

	GitFlow	GitHub Flow	Trunk-Based
Long-lived branches	main + develop	main only	main only
Branch lifetime	Days to weeks	Days	Hours to 1–2 days
Release style	Scheduled versions	Deploy after each merge	Continuous deployment
Complexity	High	Low	Low (needs strong CI & tests)
Best for	Versioned products, mobile apps, on-prem software	Web apps, small/medium teams	Mature DevOps teams, microservices

11.4 Commit message convention (Conventional Commits)

Many teams use a prefix in every commit message. Tools can then generate changelogs and version numbers automatically.

Prefix	Use for	Example
`feat:`	A new feature	feat: add redis caching layer

Prefix	Use for	Example
`fix:`	A bug fix	fix: correct nginx upstream port
`docs:`	Documentation only	docs: update deployment runbook
`chore:`	Maintenance, deps, tooling	chore: bump terraform to 1.9
`ci:`	Pipeline changes	ci: cache docker layers in build job
`refactor:`	Code change, same behaviour	refactor: split helm values per env
`feat!:`	Breaking change	feat!: drop support for API v1

CHAPTER 12

Git in the CI/CD Pipeline & GitOps

This is where Git meets DevOps. Learn how a simple git push becomes a tested, deployed application.

IN THIS CHAPTER YOU WILL LEARN

- How push → webhook → pipeline → deploy works
- The Pull Request workflow and branch protection
- A real pipeline file triggered by Git
- Git commands used inside CI jobs
- What GitOps means

In DevOps, Git is not just storage. **It is the trigger for everything.** A push, a merged Pull Request or a new tag automatically starts pipelines that build, test, scan and deploy your application.

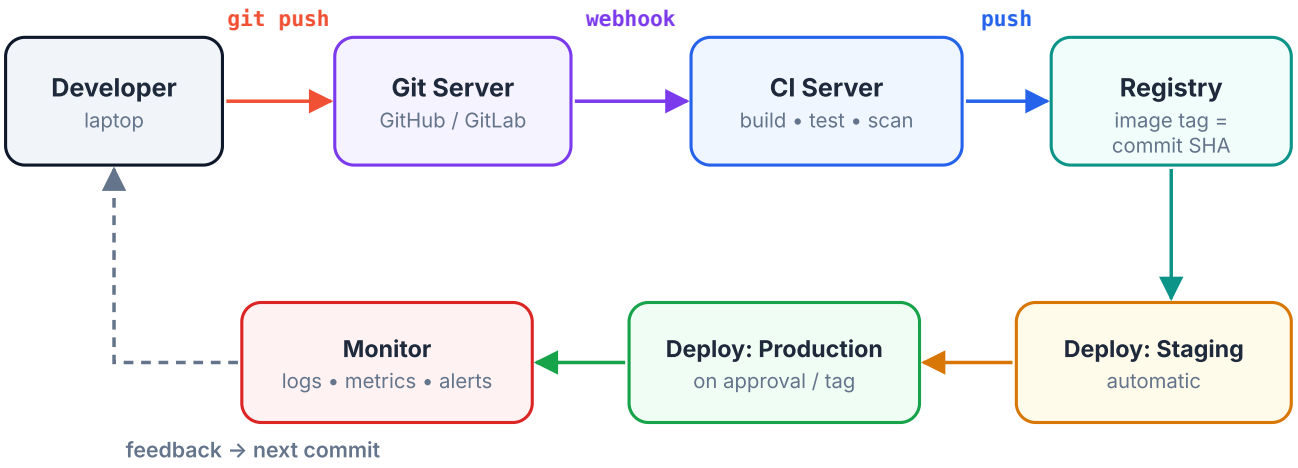


Figure 15. How a git push turns into a running application (CI/CD).

1. **Developer pushes**
   `git push` sends commits to GitHub/GitLab.
2. **Webhook fires**
   The Git server notifies the CI tool (Jenkins, GitHub Actions, GitLab CI, Azure Pipelines...).
3. **CI pipeline runs**
   It clones the repo at that exact commit, then builds, runs tests and security scans.
4. **Artifact is stored**
   A Docker image or package is pushed to a registry, tagged with the commit SHA or version tag.

5 **CD deploys**
Staging is updated automatically; production after approval or when a release tag is pushed.

6 **Feedback**
Monitoring and alerts tell the team how the change behaves, and the loop starts again.

12.1 The Pull Request workflow

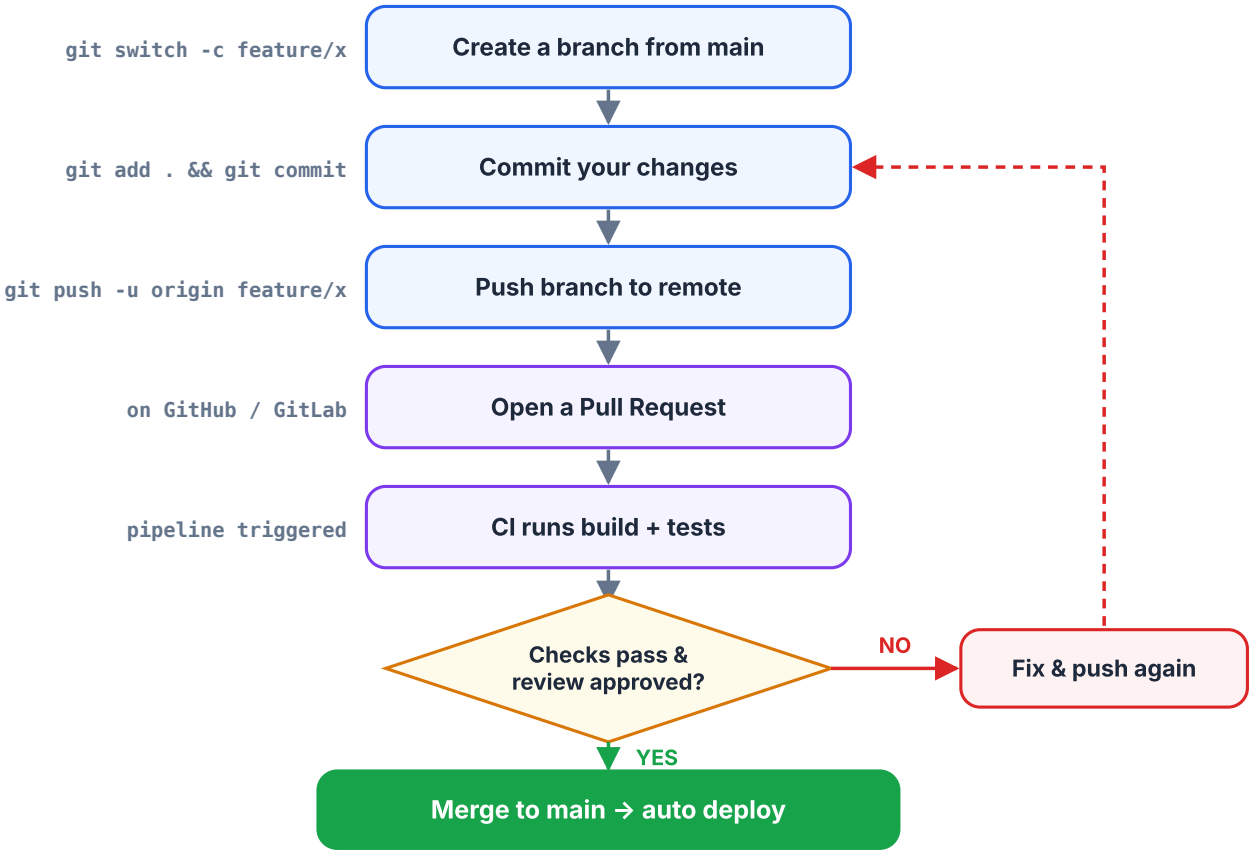


Figure 16. The Pull Request (Merge Request) workflow used in almost every DevOps team.

12.2 Protecting the main branch

On GitHub/GitLab, DevOps engineers configure **branch protection rules** so nobody can break `main`:

- ✅ No direct pushes to `main`; every change must come through a Pull Request
- ✅ At least 1–2 approvals required (often from **CODEOWNERS**)
- ✅ CI status checks must pass before merging
- ✅ Branch must be up to date with `main`
- ✅ Force pushes and branch deletion are blocked
- ✅ Optional: require signed commits and linear history

12.3 Example: a pipeline triggered by Git

.github/workflows/ci.yml

```
name: CI-CD
on:
  push:
    branches: [ main ]          # deploy when main changes
    tags: [ 'v*' ]              # release when a version tag is pushed
  pull_request:
    branches: [ main ]          # test every Pull Request

jobs:
  build-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4        # this is 'git clone' for CI
        with:
          fetch-depth: 1                 # shallow clone = faster
      - name: Run tests
        run: make test
      - name: Build Docker image tagged with the commit SHA
        run: docker build -t webapp:${{ github.sha }} .
```

12.4 Git commands you'll see inside pipelines

Command	Why pipelines use it
`git clone --depth 1 <url>`	Fast checkout of only the latest code
`git rev-parse --short HEAD`	Get the short commit SHA (e.g. `3f9c2e1`) to tag Docker images
`git describe --tags --always`	Build a version string like `v1.4.0-3-g9a9a9a9`
`git diff --name-only HEAD~1 HEAD`	Detect which services/folders changed (monorepos)
`git log -1 --pretty=%B`	Read the commit message (e.g. skip the build if it contains [skip ci])
`git rev-parse --abbrev-ref HEAD`	Get the current branch name to pick the target environment
`git tag -a vX.Y.Z && git push --tags`	Automatic release tagging

```
CI job log — build #482

ajay@devops:~/webapp$ IMAGE_TAG=$(git rev-parse --short HEAD)
ajay@devops:~/webapp$ echo $IMAGE_TAG
3f9c2e1
ajay@devops:~/webapp$ docker build -t registry.techcorp.com/webapp:$IMAGE_TAG .
ajay@devops:~/webapp$ git diff --name-only HEAD~1 HEAD
services/payment/app.py
services/payment/Dockerfile
```

12.5 GitOps: Git as the control panel for Kubernetes

In **GitOps**, the desired state of your infrastructure (Kubernetes manifests, Helm values) lives in a Git repo. A tool like **Argo CD** or **Flux** runs in the cluster and keeps it in sync with Git:

- **Deploy** = change the image tag in Git and merge the PR
- **Rollback** = `git revert` the change
- **Audit** = `git log` shows exactly who changed production, and when

> ⚙️ **In DevOps**
>
> **Git hooks** are scripts that run automatically on Git events. A `pre-commit` hook can run linters, `terraform fmt` and secret scanners before a commit is even created. The popular **pre-commit** framework makes this easy to share across the team through a `.pre-commit-config.yaml` file.

CHAPTER 13

A Day in the Life: Git in a DevOps Job

Everything you've learned, used together in one realistic working day, including an urgent production hotfix.

IN THIS CHAPTER YOU WILL LEARN

- The complete daily workflow, start to finish
- Handling an interruption with stash
- Updating a PR after review
- Tagging a release and cleaning up

Let's follow a realistic day. Your ticket **OPS-217**: *"Increase nginx upload limit to 50 MB and add a readiness probe to the Kubernetes deployment."*

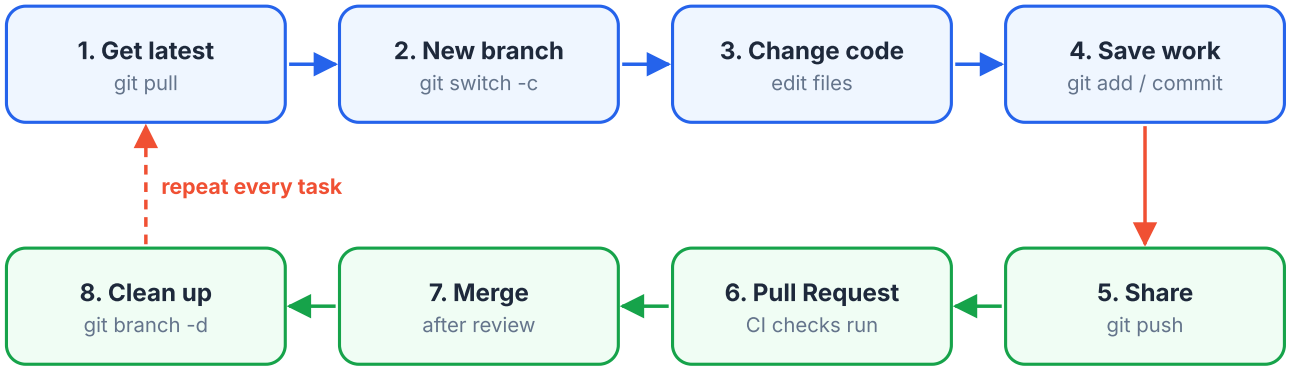


Figure 17. The daily Git loop of a DevOps engineer.

☀️ 9:30 AM: Start with fresh code

```
bash — ajay@devops: ~/webapp

ajay@devops:~/webapp$ git switch main
Switched to branch 'main'
ajay@devops:~/webapp$ git pull
Updating 3f9c2e1..8a8a8a8
Fast-forward
 terraform/providers.tf | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
```

🌿 9:35 AM: Create a branch for the ticket

```
bash — ajay@devops: ~/webapp

ajay@devops:~/webapp$ git switch -c feature/OPS-217-nginx-upload-limit
Switched to a new branch 'feature/OPS-217-nginx-upload-limit'
```

🛠️ 10:00 AM: Make changes, commit in small logical steps

```
bash — ajay@devops: ~/webapp

ajay@devops:~/webapp$ vim nginx/nginx.conf
ajay@devops:~/webapp$ git diff --stat
 nginx/nginx.conf | 1 +
 1 file changed, 1 insertion(+)
ajay@devops:~/webapp$ git add nginx/nginx.conf
ajay@devops:~/webapp$ git commit -m "feat(nginx): raise client_max_body_size to 50M"
[feature/OPS-217-nginx-upload-limit 5c5c5c5] feat(nginx): raise client_max_body_size to 50M
 1 file changed, 1 insertion(+)
ajay@devops:~/webapp$ vim k8s/deployment.yaml
ajay@devops:~/webapp$ git add k8s/deployment.yaml
ajay@devops:~/webapp$ git commit -m "feat(k8s): add readiness probe on /healthz"
[feature/OPS-217-nginx-upload-limit 6d6d6d6] feat(k8s): add readiness probe on /healthz
 1 file changed, 6 insertions(+)
```

🚀 11:00 AM: Push and open a Pull Request

```
bash — ajay@devops: ~/webapp

ajay@devops:~/webapp$ git push -u origin feature/OPS-217-nginx-upload-limit
 * [new branch]      feature/OPS-217-nginx-upload-limit -> feature/OPS-217-nginx-upload-
limit
branch 'feature/OPS-217-nginx-upload-limit' set up to track 'origin/feature/OPS-217-nginx-
upload-limit'.
# → open the PR on GitHub, CI pipeline starts automatically
```

🔥 12:15 PM: URGENT! Production hotfix needed

You have uncommitted experiments. Stash them, fix production, come back.

```
bash — ajay@devops: ~/webapp

ajay@devops:~/webapp$ git stash push -m "experimenting with probe timings"
ajay@devops:~/webapp$ git switch main && git pull
ajay@devops:~/webapp$ git switch -c hotfix/OPS-220-fix-db-timeout
ajay@devops:~/webapp$ vim config/database.yaml
ajay@devops:~/webapp$ git commit -am "fix: raise DB connection timeout to 30s"
ajay@devops:~/webapp$ git push -u origin hotfix/OPS-220-fix-db-timeout
# → fast-track PR, review, merge, pipeline deploys the fix
ajay@devops:~/webapp$ git switch feature/OPS-217-nginx-upload-limit
ajay@devops:~/webapp$ git stash pop
```

🔄 3:00 PM: Reviewer asks for a change; main has moved on

```
bash — ajay@devops: ~/webapp

ajay@devops:~/webapp$ git fetch origin
ajay@devops:~/webapp$ git rebase origin/main  # bring in the hotfix, keep history clean
Successfully rebased and updated refs/heads/feature/OPS-217-nginx-upload-limit.
ajay@devops:~/webapp$ vim k8s/deployment.yaml  # apply reviewer's suggestion
ajay@devops:~/webapp$ git commit -am "fix(k8s): set probe initialDelaySeconds to 10"
ajay@devops:~/webapp$ git push --force-with-lease  # needed after rebase
```

✅ 4:30 PM: Approved and merged. Tag the release and clean up

```
bash — ajay@devops: ~/webapp

ajay@devops:~/webapp$ git switch main && git pull
ajay@devops:~/webapp$ git tag -a v1.5.0 -m "Release 1.5.0: upload limit + readiness probe"
ajay@devops:~/webapp$ git push origin v1.5.0
 * [new tag]         v1.5.0 -> v1.5.0
ajay@devops:~/webapp$ git branch -d feature/OPS-217-nginx-upload-limit
Deleted branch feature/OPS-217-nginx-upload-limit (was 7e7e7e7).
ajay@devops:~/webapp$ git fetch --prune
```

> 🧠 **Simple Way to Remember**
>
> Notice the rhythm: **pull → branch → commit small → push → PR → merge → tag → clean up**. Teach this rhythm and your students are ready for a real DevOps job.

CHAPTER 14

Troubleshooting: Common Errors & Fixes

Error messages look scary, but most Git problems have a simple, well-known fix.

IN THIS CHAPTER YOU WILL LEARN

- The 10 errors beginners hit most often
- What each error really means
- The exact commands to fix it

Every error below is one your students **will** hit. Each comes with what it means and how to fix it.

1 `fatal: not a git repository (or any of the parent directories): .git`

Meaning: You are not inside a Git project folder.

```
cd ~/webapp     # go into the repo
# or start one:  git init
```

2 `! [rejected] main -> main (fetch first)`

Meaning: Someone pushed before you. Your local branch is behind.

```
git pull --rebase
git push
```

3 `Permission denied (publickey)`

Meaning: GitHub/GitLab doesn't recognise your SSH key.

```
ssh -T git@github.com          # test
ssh-add ~/.ssh/id_ed25519      # load key
# add id_ed25519.pub in GitHub → Settings → SSH keys
```

4 You are in 'detached HEAD' state

Meaning: You checked out a commit/tag directly, not a branch. New commits could get lost.

```
git switch main                # just go back
git switch -c my-fix           # or keep work in a new branch
```

5 I committed to main instead of my feature branch (not pushed)

Meaning: Move the commit to a new branch, then reset main.

```
git branch feature/my-work     # save the commit in a branch
git reset --hard origin/main   # reset main to the server
git switch feature/my-work
```

6 I pushed a password / API key!

Meaning: The secret is now in history. Deleting the file is NOT enough.

```
# 1. ROTATE/REVOKE the secret immediately (most important!)
# 2. Remove it from history with git filter-repo or BFG
git filter-repo --path secrets.env --invert-paths
# 3. Force push and tell the team to re-clone
```

7 Your branch and 'origin/main' have diverged

Meaning: Both you and the remote have new commits.

```
git pull --rebase     # replay your commits on top
# or: git pull --no-rebase  (merge)
```

8 error: Your local changes would be overwritten by checkout

Meaning: You have uncommitted changes that conflict with the branch you want.

```
git stash
git switch other-branch
git stash pop          # later, when you come back
```

9 `warning: LF will be replaced by CRLF`

Meaning: Windows and Linux use different line endings.

```
git config --global core.autocrlf true    # Windows
git config --global core.autocrlf input   # Linux/macOS
```

10 `remote: error: File is 312 MB; this exceeds GitHub's file size limit of 100 MB`

Meaning: Large binaries don't belong in Git.

```
git rm --cached big-file.zip
# use Git LFS or an artifact store (S3, Nexus, Artifactory)
git lfs track "*.zip"
```

CHAPTER 15

Git Cheat Sheet

Every important command on two pages. Perfect for printing, pinning or sharing.

IN THIS CHAPTER YOU WILL LEARN

- All essential commands grouped by task

⚙️ Setup

`git config --global user.name "Name"`	Set your name
`git config --global user.email "e@x.com"`	Set your email
`git config --list`	Show all settings

📁 Start

`git init`	New repo in current folder
`git clone <url>`	Copy a remote repo
`git clone --depth 1 <url>`	Shallow clone (CI)

📸 Snapshot

`git status`	What changed?
`git add <file> / .`	Stage changes
`git commit -m "msg"`	Save snapshot
`git diff / --staged`	Line-by-line changes
`git commit --amend`	Fix last commit

🌿 Branch

`git branch`	List branches
`git switch -c <name>`	Create + switch
`git switch <name>`	Switch branch
`git merge <name>`	Merge into current
`git branch -d <name>`	Delete branch

☁️ Remote

`git remote -v`	List remotes
`git fetch --prune`	Download, don't merge
`git pull`	Fetch + merge
`git push -u origin  `	First push of branch
`git push`	Upload commits

🔍 Inspect

`git log --oneline --graph`	Visual history
`git show <sha>`	Details of a commit
`git blame <file>`	Who changed each line
`git reflog`	Every move of HEAD

↩️ Undo

`git restore <file>`	Discard local edits
`git restore --staged <file>`	Unstage
`git reset --soft HEAD~1`	Undo commit, keep changes
`git revert <sha>`	Safe undo (shared branch)
`git clean -fd`	Delete untracked files

🧰 Advanced

`git stash / pop`	Shelve work
`git rebase main`	Replay on top of main
`git rebase -i HEAD~3`	Squash/edit commits
`git cherry-pick <sha>`	Copy one commit
`git tag -a v1.0 -m ""`	Release tag
`git bisect start`	Find bad commit

> 💡 **Pro Tip**
>
> Print this chapter and stick it next to your monitor. After two weeks you won't need it anymore.

CHAPTER 16

Top 20 Git Interview Questions for DevOps

The questions that come up again and again in DevOps, SRE and Cloud interviews, with short, clear answers.

IN THIS CHAPTER YOU WILL LEARN

- How to explain core Git concepts in an interview
- Scenario questions: rollback, leaked secrets, lost commits

Q1 What is the difference between Git and GitHub?

Git is the version control tool that runs locally. GitHub is a hosting service for Git repositories that adds collaboration features like Pull Requests, issues and Actions (CI/CD).

Q2 What is the difference between `git fetch` and `git pull` ?

`fetch` only downloads new data and updates remote-tracking branches; your files are untouched. `pull` = `fetch` + `merge` (or rebase), so it also updates your current branch.

Q3 Merge vs rebase?

Merge combines branches with a merge commit and keeps the true history. Rebase replays your commits on top of another branch for a linear history, but rewrites commit IDs, so never rebase shared/public branches.

Q4 `git reset` vs `git revert` ?

Reset moves the branch pointer backwards (rewrites history), so it's fine for local, unpushed work. Revert creates a new commit that undoes an old one; it is safe for shared branches and is the standard production rollback.

Q5 What is HEAD? What is a detached HEAD?

HEAD points to your current location, normally a branch. Detached HEAD means HEAD points directly to a commit (e.g. after checking out a tag), so new commits don't belong to any branch unless you create one.

Q6 What is the staging area and why is it useful?

It's a middle layer between your working files and the repository. It lets you choose exactly which changes go into the next commit, which gives you small, focused commits.

Q7 **How do you resolve a merge conflict?**

Run `git status` to find conflicted files, edit them to remove the `<<<<<<<`, `=======`, `>>>>>>>` markers and keep the right code, then `git add` and `git commit`.

Q8 **What is `git stash` ?**

It temporarily shelves uncommitted changes so you can switch context (e.g. for a hotfix), and later restore them with `git stash pop`.

Q9 **What is cherry-pick and when do you use it?**

It applies one specific commit from another branch onto the current one. It's common for backporting a bug fix to a release branch.

Q10 **How do you undo the last commit?**

Not pushed: `git reset --soft HEAD~1` (keep changes) or `--hard` (discard). Already pushed: `git revert HEAD`.

Q11 **A file is in .gitignore but Git still tracks it. Why?**

.gitignore only works for untracked files. Run `git rm --cached <file>` and commit.

Q12 **Lightweight vs annotated tag?**

A lightweight tag is just a name pointing to a commit. An annotated tag (`-a`) is a full object with author, date and message; use it for releases.

Q13 **Fork vs clone?**

A fork is a server-side copy of a repo under your account (GitHub feature). A clone is a local copy on your machine (Git feature).

Q14 **`--force` vs `--force-with-lease` ?**

`--force` overwrites the remote branch blindly. `--force-with-lease` refuses if someone else pushed new commits since your last fetch, so it is much safer.

Q15 **You accidentally pushed a secret. What do you do?**

First revoke/rotate the secret immediately. Then remove it from history (git filter-repo / BFG), force-push, ask the team to re-clone, and add secret scanning to prevent it again.

Q16 **How did you recover lost commits?**

With `git reflog`: find the commit SHA from before the mistake and run `git reset --hard <sha>` or create a branch from it.

Q17 **What branching strategy have you used?**

Describe one clearly, e.g. GitHub Flow: protected main, short feature branches, PRs with required reviews and CI checks, auto-deploy on merge, tags for releases.

Q18 **How does Git integrate with CI/CD?**

A push or PR triggers a webhook to the CI server, which checks out that exact commit, builds and tests it, publishes an artifact tagged with the commit SHA or version tag, and deploys it.

Q19 **What is `git bisect` ?**

A binary search through history to find the commit that introduced a bug. You mark commits good/bad and Git narrows it down in log2(n) steps.

Q20 **What is GitOps?**

An approach where Git is the single source of truth for infrastructure and deployments. A controller (Argo CD, Flux) continuously syncs the cluster to what's in Git.

CHAPTER 17

Hands-On Practice Labs

Ten practical exercises, from your first commit to a tagged release. Perfect as homework or a classroom workshop.

IN THIS CHAPTER YOU WILL LEARN

- Practising every major command in a safe playground

Reading is not enough. Do these 10 hands-on exercises in order. Each one builds on the previous. **Hint** shows the commands you'll need.

LAB 01

Your first repo

Create a folder `git-lab`, initialise Git, add a `README.md` and make your first commit.

Hint: `git init · git add · git commit`

LAB 02

Read the history

Make 3 more commits changing the README. Show the history in one line per commit, then show the details of the second commit.

Hint: `git log --oneline · git show`

LAB 03

Stage selectively

Change two files, but commit only one of them. Then commit the other with a different message.

Hint: `git add <file> · git status`

LAB 04

Branch & merge

Create `feature/about-page`, add `about.html`, commit, switch back to main and merge it.

Hint: `git switch -c · git merge`

LAB 05

Create a conflict on purpose

On two different branches, change the **same line** of README differently. Merge them and resolve the conflict.

Hint: `git merge · edit · git add · git commit`

LAB 06

Push to GitHub

Create an empty repo on GitHub, connect it with SSH and push main and your feature branch.

Hint: `ssh-keygen · git remote add · git push -u`

LAB 07

Undo like a pro

Make a bad commit and undo it with reset (keep changes). Then push a commit and undo it with revert.

Hint: `git reset --soft · git revert`

LAB 08

Interrupt & resume

Start editing a file, stash it, create a hotfix branch, commit a fix, return and pop the stash.

Hint: `git stash · git stash pop`

LAB 09

Rescue mission

Run `git reset --hard HEAD~2`, then recover both 'lost' commits.

Hint: `git reflog · git reset --hard <sha>`

LAB 10

Release it

Add a `.gitignore` that ignores `*.log` and `.env`, tag the commit `v1.0.0` and push the tag.

Hint: `git tag -a · git push origin v1.0.0`

💡 **Bonus Challenge**

Challenge for advanced students: fork an open-source project, fix a typo in its docs, and open a real Pull Request. It's the best way to practise the full collaboration workflow.



You now speak Git.

You have learned how Git works inside, every command a DevOps engineer uses day to day, how teams collaborate with branches and Pull Requests, and how Git drives CI/CD and GitOps.

Next steps: practise the labs until they feel automatic, then explore CI/CD tools (GitHub Actions, GitLab CI, Jenkins) and GitOps tools (Argo CD, Flux).

If this book helped you, share it with someone who is starting their DevOps journey. ♻️

— Ajay Upadhyay

DevOps Engineer